

PYTHON DESDE CERO

EDICIÓN 2026 · NIVELES 1—7

El lenguaje que mueve la inteligencia artificial del mundo, explicado desde la primera línea: variables, tipos de datos, operadores, print y f-strings, y todas las formas de tomar decisiones (if, elif, ternario y match). Con decenas de ejemplos de código a color, explicados línea por línea.

NIVEL 1

¿Qué es Python y por qué está en todas partes?

NIVEL 2

print(): tu primer programa

NIVEL 3

Variables y tipos de datos

NIVEL 4

input(), conversión y f-strings

NIVEL 5

Operadores: las herramientas de cálculo

NIVEL 6

Decisiones: if, elif, ternario y match

NIVEL 7

Errores, depuración y proyecto final

PARA ESTUDIANTES QUE ESCRIBEN SU PRIMERA LÍNEA DE CÓDIGO · MODO HISTORIA: JUGAR EN ORDEN

MAPA DEL CURSO

Este es el tomo donde por fin escribes código de verdad. Todo lo que aprendiste sobre algoritmos (pasos, decisiones, variables) tiene ahora un idioma real donde vivir. Siete niveles, decenas de ejemplos, cero relleno.

1	¿Qué es Python? Entender el lenguaje y cómo se ejecuta	EMPIEZA AQUÍ
2	print() y tu primer programa Hacer que la máquina te hable	ESTE TOMO
3	Variables y tipos de datos Guardar información y saber qué es cada cosa	ESTE TOMO
4	input(), conversión y f-strings Conversar con el usuario y dar formato	ESTE TOMO
5	Operadores Calcular, comparar y combinar condiciones	ESTE TOMO
6	Decisiones if, elif, else, ternario y match	ESTE TOMO
7	Errores y proyecto final Depurar como un pro y construir algo tuyo	JEFE FINAL

CÓMO USAR ESTE TOMO

- 1. No leas el código: ESCRÍBELO** — copiar a mano cada ejemplo (sin copiar y pegar) es lo que instala el lenguaje en tus dedos.
- 2. Rómpelo a propósito** — cambia un valor, borra una comilla, mira qué error sale. Los errores son el profesor más honesto.
- 3. Pasa el Checkpoint** — si puedes explicarlo con tus palabras y sin mirar, el nivel es tuyo.

¿QUÉ ES PYTHON?

El lenguaje que mueve la IA del mundo... y se lee casi como inglés

LO QUE DESBLOQUEARÁS

- ▶ Explicar qué es Python y qué lo hace distinto de otros lenguajes.
- ▶ Entender por qué es el rey indiscutible de la inteligencia artificial.
- ▶ Saber cómo se ejecuta tu código: intérprete, archivos .py y consola.
- ▶ Conocer la filosofía que hay detrás de su diseño.

PALABRAS CLAVE

Python intérprete legibilidad indentación
multiparadigma librería PyPI REPL Zen
de Python

1.1 Un lenguaje diseñado para humanos

Antes de cualquier definición, mira esto. Este es un programa completo en Python que decide si aprobaste:

primer_vistazo.py

```
nota = 4.2
if nota >= 3.0:
    print("¡Aprobaste!")
else:
    print("A recuperar")
```

SALIDA EN PANTALLA

¡Aprobaste!

Léelo en voz alta: "nota es 4.2; si nota es mayor o igual a 3.0, imprime ¡Aprobaste!; si no, imprime A recuperar". **Casi lo entendiste sin saber Python**, y eso no es casualidad: es exactamente el objetivo con el que fue diseñado. Su creador, el neerlandés **Guido van Rossum**, lo publicó en 1991 con una obsesión: que el código fuera fácil de LEER, porque el código se escribe una vez y se lee cientos.

CONCEPTO CLAVE

Python: lenguaje de programación de alto nivel, interpretado, multiparadigma y de propósito general, famoso por su sintaxis clara y su enorme colección de librerías. "Propósito general" significa que sirve para casi todo: no está encerrado en un solo tipo de tarea.

Hay un detalle único en ese primer ejemplo que conviene señalar desde ya: **los espacios en blanco del inicio de la línea 3 NO son decoración**. En la mayoría de lenguajes, los bloques se marcan con llaves { }; en Python se marcan con **indentación** (sangría). Esa línea está "dentro" del if porque está corrida hacia la derecha. Es la característica más famosa (y más amada u odiada) del lenguaje, y la estudiarás a fondo en el Nivel 6.

1.2 Por qué Python está en todas partes en 2026

Python lleva años encabezando los rankings de lenguajes más usados del mundo, y no por moda. La razón principal tiene nombre: **las librerías**, que son paquetes de código ya escrito por otras personas que puedes usar en tu programa. Si necesitas analizar datos, hacer gráficos o entrenar un modelo de inteligencia artificial, alguien ya construyó la herramienta y la publicó gratis. Hay cientos de miles de paquetes disponibles en el repositorio oficial (PyPI).

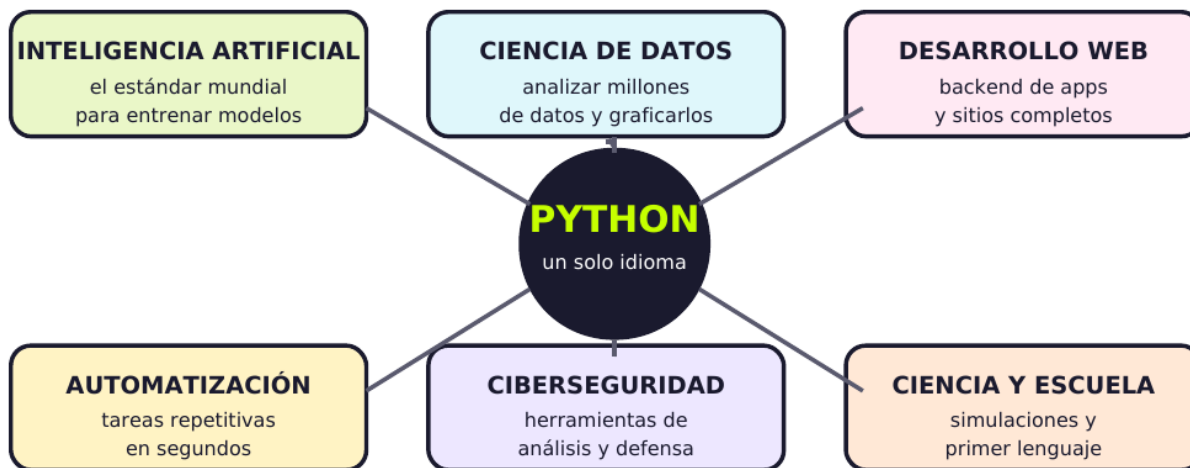


FIG. 1.1 — Un solo lenguaje, seis mundos. Aprender Python no te encierra en un camino: te abre la puerta a la IA, al análisis de datos, al backend web, a la automatización y a la investigación científica.

El caso de la **inteligencia artificial** merece párrafo aparte, porque es la razón por la que este lenguaje explotó. Prácticamente todas las herramientas serias de IA (las que entrenan los modelos que usas a diario) se manejan desde Python. Curiosidad que sorprende a todo el mundo: por dentro, esas librerías están escritas en lenguajes rapidísimos como C++, pero **te dejan manejarlas desde Python porque es cómodo para los humanos**. Python es el volante; C++ es el motor. Y aprender a manejar es lo que abre las puertas.

1.3 Cómo se ejecuta tu código

Python es un lenguaje **interpretado**: un programa llamado intérprete lee tu código línea por línea y lo va ejecutando al instante. No hay que "compilar" ni esperar: escribes y corre. Por eso es tan bueno para aprender, para experimentar y para equivocarse rápido.

Forma de ejecutarlo	Cuándo la usarás
La consola interactiva (REPL)	Escribes una línea y ves el resultado al instante. Perfecta para probar ideas sueltas: "¿cuánto da 7 // 2?". Se abre escribiendo python en la terminal.
Un archivo .py	Escribes el programa completo en un archivo y lo ejecutas entero. Es la forma normal de trabajar en proyectos reales.
Un editor con botón de ejecutar	Lo mismo, pero cómodo: escribes en VS Code (o similar) y ejecutas con un clic viendo la salida en la terminal integrada.
Un cuaderno (notebook)	Bloques de código con resultados y texto mezclados. Es el formato favorito en ciencia de datos e IA.

Sobre las versiones: la serie actual es **Python 3**, y el equipo publica una versión nueva cada octubre (3.10, 3.11, 3.12, 3.13...). Todo lo que aprendas en este tomo funciona en cualquier versión moderna, con una sola excepción que te avisaré: **la estructura match, que existe desde Python 3.10 en adelante**. Si alguna vez ves código antiguo con print sin paréntesis, es Python 2: quedó oficialmente retirado en 2020, y ya no debes aprenderlo.

EN LA VIDA REAL · PYTHON YA TRABAJA PARA TI (AUNQUE NO LO VEAS)

Las recomendaciones que consumes —qué video sigue, qué canción va después, qué producto te aparece— salen de modelos entrenados casi siempre con herramientas de Python. Tu feed es, en buena parte, Python ejecutándose en un servidor lejano.

Los científicos que fotografiaron un agujero negro procesaron sus datos con librerías de Python. También lo usa la NASA, el CERN y prácticamente cualquier laboratorio que analice datos hoy.

Y en lo cotidiano: mucha gente usa Python para automatizar aburrimientos: renombrar 500 fotos, descargar sus notas de una página, ordenar carpetas, avisar por mensaje cuando baja el precio de unas zapatillas. Ese es probablemente el primer superpoder útil que vas a estrenar.

? ¿SABÍAS QUÉ?

Python trae escondido un poema. Si escribes **import this** en la consola, aparece "El Zen de Python": 19 principios de diseño escritos por Tim Peters en 1999, mitad filosofía y mitad chiste interno. Entre ellos: "lo bello es mejor que lo feo", "lo explícito es mejor que lo implícito", "lo simple es mejor que lo complejo" y el famoso "la legibilidad cuenta". El detalle más gracioso: la lista dice que son 20 principios, pero solo hay 19 escritos... el número 20 nunca se publicó, y la broma es que sigue pendiente. Es una de las pocas veces en la historia en que **un lenguaje de programación incluye su propia declaración de valores dentro del propio lenguaje**, y explica por qué la comunidad de Python es tan obsesiva con escribir código claro.

RETO DE OBSERVADOR

MISIÓN: "PRIMER CONTACTO" · 20 MIN

1. Consigue un lugar donde ejecutar Python: instala Python y un editor, o usa un intérprete en línea gratuito desde el navegador (busca "python online"). Escribe el ejemplo de 1.1 y ejecútalo.
2. Cambia el valor de nota a 2.5 y vuelve a ejecutar. **Rómpelo a propósito:** ahora borra los 4 espacios de sangría de la línea del print y mira el error que aparece. Anótalo: es tu primer IndentationError.
3. Escribe en la consola interactiva `import this` y lee el Zen de Python. Elige el principio que más te guste y explica en una frase qué crees que significa.
4. **Nivel jefe:** investiga tres programas o servicios famosos que usen Python y anota para qué lo usan. Bonus si encuentras uno que te sorprenda.

Recompensa desbloqueada: entorno listo. Ya tienes dónde escribir y ejecutar todo lo que viene.

CHECKPOINT · NIVEL 1

1. ¿Qué significa que Python sea "interpretado" y qué ventaja tiene para quien está aprendiendo?
2. ¿Por qué Python domina en inteligencia artificial si por dentro las librerías usan C++?
3. ¿Qué son la indentación y las librerías, y por qué son tan importantes en este lenguaje?

ESTE NIVEL EN 3 IDEAS

1. **Python fue diseñado para que el código sea fácil de leer: por eso se parece tanto al inglés.**
2. **Su fuerza real son las librerías: miles de herramientas ya construidas listas para usar.**
3. **Es interpretado (escribes y corre) y usa indentación en vez de llaves para marcar bloques.**

PRINT(): HACER HABLAR A LA MÁQUINA

La función que usarás miles de veces en tu vida

LO QUE DESBLOQUEARÁS

- ▶ Escribir y entender tu primer programa completo.
- ▶ Dominar `print()` con varios argumentos y sus opciones `sep` y `end`.
- ▶ Usar comillas, saltos de línea y caracteres especiales sin errores.
- ▶ Comentar tu código como lo hacen los profesionales.

PALABRAS CLAVE

función	argumento	string	comillas
salto de línea <code>\n</code>		<code>sep</code>	<code>end</code> comentario <code>#</code>
consola			

2.1 Anatomía de la línea más famosa del mundo

Existe una tradición sagrada: el primer programa que se escribe en cualquier lenguaje muestra el mensaje "Hola, mundo". Vamos a diseccionarlo pieza por pieza, porque cada símbolo tiene un nombre y una razón:



FIG. 2.1 — Cada pieza tiene nombre. `print` es la función (la orden), los paréntesis delimitan lo que le entregas, y las comillas convierten esas palabras en un texto que Python debe mostrar tal cual.

CONCEPTO CLAVE

Función: una orden con nombre que hace un trabajo por ti. Se ejecuta escribiendo su nombre seguido de paréntesis. Lo que va dentro de los paréntesis se llama **argumento**: la información que le entregas para que trabaje. `print()` es una función que recibe lo que quieras y lo muestra en la consola.

2.2 Textos (strings): comillas y sus trampas

Todo lo que va entre comillas es un **string** (cadena de texto). Python acepta comillas dobles o simples, sin diferencia... siempre que abras y cierres con las mismas. Y eso resuelve un problema práctico:

`comillas.py`

```
print("Hola, mundo")
print('También funciona con comillas simples')

# ¿Y si el texto tiene un apóstrofo?
print("Hoy sí es día de código")

# Comillas dobles dentro de un texto: usa las simples por fuera
print('Ella dijo: "esto está fácil"')
```

SALIDA EN PANTALLA

```
Hola, mundo
También funciona con comillas simples
Hoy sí es día de código
Ella dijo: "esto está fácil"
```

Hay caracteres que no se pueden teclear directamente dentro de un texto, y para ellos existen las **secuencias de escape**: combinaciones que empiezan con una barra invertida. Las dos que usarás siempre:

escapes.py

```
print("Primera línea\nSegunda línea") # \n = salto de línea
print("Nombre:\tAna")                 # \t = tabulación
print("Ella dijo: \"hola\"")           # \" = comilla literal
```

SALIDA EN PANTALLA

```
Primera línea
Segunda línea
Nombre:      Ana
Ella dijo: "hola"
```

2.3 print() con varios argumentos: sep y end

print() puede recibir varias cosas separadas por comas y las muestra todas seguidas, poniendo un espacio entre ellas automáticamente. Además tiene dos opciones muy útiles: **sep** cambia ese separador y **end** cambia lo que va al final (por defecto, un salto de línea):

print_avanzado.py

```
print("Ana", "Luis", "Sofía")
print("Ana", "Luis", "Sofía", sep=" - ")
print("2026", "08", "07", sep="/")

print("Cargando", end="")
print("...", end="")
print("listo")
```

SALIDA EN PANTALLA

```
Ana Luis Sofía
Ana - Luis - Sofía
2026/08/07
Cargando...listo
```

Fíjate en el último bloque: las tres líneas se imprimieron **en el mismo renglón** porque end="" reemplazó el salto de línea por nada. Ese truco es el que usan las barras de progreso de los programas de consola.

2.4 Comentarios: escribirle notas a tu yo del futuro

Todo lo que va después de una almohadilla (#) en una línea es un **comentario**: Python lo ignora completamente. Sirve para explicar POR QUÉ hiciste algo (no QUÉ hiciste: eso ya lo dice el código). Y aquí va un consejo que vale oro: **el principal lector de tus comentarios serás tú mismo dentro de tres meses, cuando ya no recuerdes nada.**

comentarios.py

```
# Programa de bienvenida - versión 1
print("Bienvenido al sistema") # este comentario va al final de la línea

# print("Esta línea NO se ejecuta")
# Comentar código temporalmente es un truco clásico para probar cosas
```

SALIDA EN PANTALLA

Bienvenido al sistema

ERROR CLÁSICO · LOS 3 TROPIEZOS DEL PRIMER DÍA

- 1. Olvidar cerrar comillas o paréntesis:** Python responde con `SyntaxError`. Casi siempre el problema está en la línea que menciona el error... o en la anterior.
- 2. Escribir Print o PRINT:** Python distingue mayúsculas de minúsculas. `print` y `Print` son dos cosas distintas, y la segunda no existe (`NameError`).
- 3. Mezclar tipos de comillas:** abrir con doble y cerrar con simple no funciona. Abre y cierra con la misma.

EN LA VIDA REAL · PRINT() ES LA HERRAMIENTA MÁS USADA DEL PLANETA

Los **programadores profesionales** usan `print()` todos los días, no solo para mostrar resultados sino para **espíar su propio programa**: ponen prints en medio del código para ver qué valor tiene una variable en ese punto exacto. Es la versión rápida de la prueba de escritorio, y tiene nombre cariñoso: "depurar con prints".

Los **mensajes que ves en las apps** —"Cargando...", "Guardado con éxito", "Error de conexión"— son la misma idea con maquillaje: alguien decidió qué texto mostrarle al usuario y en qué momento. Comunicar bien con el usuario empieza aquí.

? ¿SABÍAS QUÉ?

En Python 2, `print` NO era una función: era una palabra reservada y se escribía sin paréntesis (`print "Hola"`). Cuando salió Python 3 en 2008, convertirla en función fue uno de los cambios más polémicos, porque **rompió absolutamente todos los programas escritos hasta entonces**. La migración fue tan dolorosa que las dos versiones convivieron **doce años**: Python 2 solo se retiró oficialmente el 1 de enero de 2020, con una cuenta regresiva pública incluida. ¿Por qué hacer un cambio tan caro? Porque como función, `print` gana superpoderes: acepta opciones como `sep` y `end`, se puede pasar a otras funciones y se comporta como todo lo demás en el lenguaje. **Lección de ingeniería: a veces vale la pena romper para arreglar bien, pero el costo se paga durante una década.**

RETO DE OBSERVADOR

MISIÓN: "LA CONSOLA HABLA" · 25 MIN

1. Escribe un programa que se presente en 4 líneas: tu nombre, tu edad, tu comida favorita y una meta. Usa al menos una vez `\n` para producir dos renglones con un solo `print`.
2. Dibuja algo con texto: un cuadrado, un triángulo o tu inicial usando varios `print` con asteriscos. Aquí descubrirás por qué los espacios importan.
3. Usa `sep` y `end` para imprimir una fecha con barras (2026/08/07) y una cuenta regresiva en un solo renglón (3... 2... 1... ¡Ya!).
4. **Nivel jefe:** provoca los tres errores del recuadro rojo a propósito, uno por uno, y copia el mensaje exacto que devuelve Python. Tener ese "diccionario de errores" propio te ahorrará horas en los próximos niveles.

Recompensa desbloqueada: voz propia. Tu programa ya puede comunicarse con el mundo exterior.

CHECKPOINT · NIVEL 2

1. ¿Qué diferencia hay entre la función `print`, sus paréntesis y su argumento?
2. ¿Para qué sirven `sep` y `end`? Da un ejemplo de cada uno.
3. ¿Por qué los comentarios deben explicar el "por qué" y no el "qué"?

ESTE NIVEL EN 3 IDEAS

1. **`print()` es una función:** recibe argumentos entre paréntesis y los muestra en la consola.
2. **Los textos van entre comillas (dobles o simples, pero coherentes) y admiten escapes como `\n` y `\t`.**
3. **Los comentarios con `#` los ignora Python y los agradece tu yo del futuro.**

VARIABLES Y TIPOS DE DATOS

Guardar información y saber exactamente qué es cada cosa

LO QUE DESBLOQUEARÁS

- ▶ Crear variables y entender qué significa realmente el signo `=`.
- ▶ Distinguir los 4 tipos básicos: `int`, `float`, `str` y `bool`.
- ▶ Nombrar variables como un profesional (y evitar los nombres prohibidos).
- ▶ Entender el tipado dinámico y fuerte de Python, con sus trampas.

PALABRAS CLAVE

variable	asignación	int	float	str	bool
<code>type()</code>	<code>snake_case</code>	palabra reservada			
constante					

3.1 Variables: cajas con etiqueta

Una **variable** es un espacio en la memoria con un nombre, donde guardas un dato para usarlo después. La imagen mental correcta es una caja con etiqueta. Y aquí va la aclaración más importante del nivel: **el signo `=` NO significa "es igual a"**: significa "guarda esto aquí". Se lee de derecha a izquierda: toma el valor de la derecha y mételo en la caja de la izquierda.

LOS 4 TIPOS BÁSICOS: CAJAS QUE GUARDAN COSAS DISTINTAS



CUIDADO CON LAS COMILLAS: cambian el TIPO, y el tipo cambia TODO

<code>edad = 17</code>	número: se puede sumar, restar, comparar
<code>edad = "17"</code>	texto: "17" + "3" da "173", no 20

FIG. 3.1 — Los cuatro tipos básicos y la asignación. En Python no declaras el tipo: él lo deduce solo del valor que le pones.

variables.py

```
nombre = "Sofía"    # se crea la caja y se guarda un texto
edad = 17           # un número entero
promedio = 4.35     # un número con decimales
es_estudiante = True # verdadero o falso
```

```
print(nombre, edad, promedio, es_estudiante)
```

```
edad = 18           # el valor viejo se REEMPLAZA
print("Ahora tiene", edad)
```

SALIDA EN PANTALLA

```
Sofía 17 4.35 True
Ahora tiene 18
```

Fíjate en el final: al asignar un valor nuevo, el anterior desaparece para siempre. Por eso se llaman "variables": su contenido varía. En una caja solo cabe un valor a la vez.

3.2 Los cuatro tipos básicos

Tipo	Qué guarda	Ejemplos	Ojo con esto
int	Números enteros	17 0 -5 1000000	Sin decimales y sin comas ni puntos de miles.
float	Números con decimales	4.35 -0.5 3.0	Se usa PUNTO decimal, nunca coma: 4.5 sí, 4,5 no.
str	Texto (cadenas)	"Ana" "17" "¡Hola!"	"17" entre comillas es TEXTO, no número. Trampa clásica.
bool	Verdadero o falso	True False	Con mayúscula inicial siempre: true en minúscula da error.

Para preguntarle a Python de qué tipo es algo existe la función **type()**, y es tu mejor amiga cuando algo no funciona y no sabes por qué:

```
type.py

print(type(17))
print(type(4.35))
print(type("17"))
print(type(True))

SALIDA EN PANTALLA
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

3.3 Nombrar variables: reglas y buenas costumbres

REGLAS OBLIGATORIAS (SI LAS ROMPES, ERROR)

- 1. No pueden empezar con un número:** nota1 está bien; 1nota da error.
- 2. Sin espacios ni tildes ni ñ:** usa guion bajo. mi nota no funciona; mi_notasí. (La ñ técnicamente funciona, pero la comunidad recomienda evitarla.)
- 3. Distinguen mayúsculas:** edad, Edad y EDAD son tres variables diferentes.
- 4. No pueden ser palabras reservadas:** if, else, for, while, True, class, import y unas 30 más ya tienen dueño.

Y ahora las buenas costumbres, que no son obligatorias pero te delatan como novato si no las sigues. La comunidad de Python usa **snake_case**: todo en minúsculas y palabras separadas por guion bajo (promedio_final, nombre_completo). Además, **el nombre debe explicar el contenido**: "x", "a1" o "dato" no dicen nada; "promedio_final" lo dice todo. Existe una convención más: los valores que nunca deben cambiar se escriben en MAYÚSCULAS (PI = 3.1416, NOTA_MINIMA = 3.0). Python no te lo impide, pero avisa a los demás: "esto no se toca".

3.4 Tipado dinámico y fuerte: la personalidad de Python

Python tiene dos características que suenan a jerga y se entienden en un minuto:

tipado.py

```
# DINÁMICO: la misma caja puede cambiar de tipo
dato = 17      # ahora es int
dato = "hola"  # ahora es str, y a Python le parece bien

# FUERTE: pero NO mezcla tipos a la loca
edad = 17
texto = "años"
print(edad + texto) # ¡ERROR!
```

SALIDA EN PANTALLA

```
TypeError: unsupported operand type(s) for +:
'int' and 'str'
```

Esa última línea produce el error más frecuente de todo principiante, y conviene entenderlo bien porque es una buena noticia disfrazada: Python **prefiere avisarte con un error antes que adivinar mal**. Otros lenguajes habrían inventado un resultado raro sin decir nada. La solución llega en el Nivel 4 (convertir tipos) y también se puede resolver así:

solucion.py

```
edad = 17
print("Tengo", edad, "años")      # con comas, print acepta de todo
print("Tengo " + str(edad) + " años") # o conviertes el número a texto
```

SALIDA EN PANTALLA

```
Tengo 17 años
Tengo 17 años
```

Un vistazo adelantado: cuando un dato no basta

Cuando necesites guardar VARIOS datos en una sola caja, Python tiene estructuras que estudiarás más adelante, pero conviene que reconozcas sus nombres: **listas** (ordenadas y modificables, notas = [4.0, 3.5, 5.0]), **tuplas** (como listas pero inmutables) y **diccionarios** (parejas de clave y valor: {"nombre": "Ana", "edad": 17}). Por ahora basta con saber que existen y que se construyen sobre lo que ya sabes.

EN LA VIDA REAL · VARIABLES POR TODAS PARTES

Tu perfil en cualquier app es un montón de variables: nombre (str), edad (int), seguidores (int), cuenta_verificada (bool), calificación_promedio (float). Cuando cambias tu foto o tu bio, un programa está reasignando valores exactamente como en el ejemplo de 3.1.

El marcador de un videojuego vive en una variable que se actualiza cientos de veces por partida (puntos = puntos + 10). Tu vida, tu munición y tu posición son variables actualizándose 60 veces por segundo.

? ¿SABÍAS QUÉ?

Escribe esto en cualquier consola de Python: **print(0.1 + 0.2)**. El resultado no es 0.3, sino **0.30000000000000004**. No es un bug de Python: pasa igual en JavaScript, Java, C y casi cualquier lenguaje. La razón es que las computadoras guardan los decimales en binario, y algunos números (como 0.1) no tienen representación exacta en base 2, igual que 1/3 no se puede escribir exacto en decimal (0.3333...). El error es minúsculo, pero real, y explica por qué **los sistemas bancarios NUNCA usan float para el dinero** (usan tipos especiales de precisión exacta o guardan los centavos como enteros). Recuerda esto cuando compares decimales: preguntar si dos floats son exactamente iguales es buscarse problemas.

RETO DE OBSERVADOR

MISIÓN: "TU FICHA DE PERSONAJE" · 25 MIN

1. Crea 6 variables que te describan usando los cuatro tipos al menos una vez (nombre, edad, estatura, tiene_mascota, videojuego_favorito, horas_de_sueño). Imprímelas todas y luego imprime el `type()` de cada una.
2. Escribe tres nombres de variable MALOS (poco claros) y sus versiones buenas. Ejemplo: `n` → `nombre_completo`. Explica por qué la segunda es mejor.
3. Provoca el `TypeError` de 3.4 a propósito y luego arrégalo de las dos formas mostradas. Explica con tus palabras por qué Python se queja.
4. **Nivel jefe:** ejecuta `print(0.1 + 0.2)` y luego prueba `print(round(0.1 + 0.2, 2))`. Investiga qué hace `round()` y explica por qué resuelve el problema... aunque no elimine su causa.

Recompensa desbloqueada: memoria propia. Tus programas ya pueden recordar cosas en vez de solo repetir textos fijos.

CHECKPOINT · NIVEL 3

1. ¿Qué hace exactamente el signo `=` y por qué es incorrecto leerlo como "igual"?
2. Nombra los 4 tipos básicos con un ejemplo de cada uno y una trampa asociada.
3. ¿Qué significa que Python sea de tipado dinámico pero fuerte?

ESTE NIVEL EN 3 IDEAS

1. Una variable es una caja con etiqueta: `=` guarda un valor y el anterior se pierde.
2. `int`, `float`, `str` y `bool` son los cimientos; `type()` te dice cuál es cuál cuando dudas.
3. Los nombres claros en `snake_case` no son estética: son la mitad de la legibilidad de tu código.

INPUT(), CONVERSIÓN Y F-STRINGS

Que tu programa pregunte, entienda y responda con estilo

LO QUE DESBLOQUEARÁS

- ▶ Pedirle datos al usuario con `input()` y guardarlos en variables.
- ▶ Convertir tipos con `int()`, `float()` y `str()` sin morir en el intento.
- ▶ Dominar las f-strings: la forma moderna de dar formato a los textos.
- ▶ Controlar decimales y alineación para que tu salida se vea profesional.

PALABRAS CLAVE

<code>input()</code>	<code>casting</code>	<code>int()</code>	<code>float()</code>	<code>str()</code>
<code>ValueError</code>	<code>f-string</code>	<code>interpolación</code>	<code>:.2f</code>	

4.1 input(): la puerta de entrada

Hasta ahora tus programas solo hablaban. Con **input()** empiezan a escuchar: la función muestra un mensaje, espera a que la persona escriba algo y presione Enter, y devuelve lo que escribió para que lo guardes en una variable.

saludo.py

```
nombre = input("¿Cómo te llamas? ")
print("¡Mucho gusto,", nombre + "!")
```

SALIDA EN PANTALLA

```
¿Cómo te llamas? Sofía
¡Mucho gusto, Sofía!
```

Y aquí llega **la trampa número uno de todo principiante en Python**, tan famosa que merece su propia figura: sin importar lo que la persona escriba —un número, una letra, lo que sea—, **input()** SIEMPRE devuelve un string. Siempre. Aunque teclee 5, lo que llega a tu variable es el texto "5".

LA TRAMPA MÁS FAMOSA: input() SIEMPRE DEVUELVE TEXTO

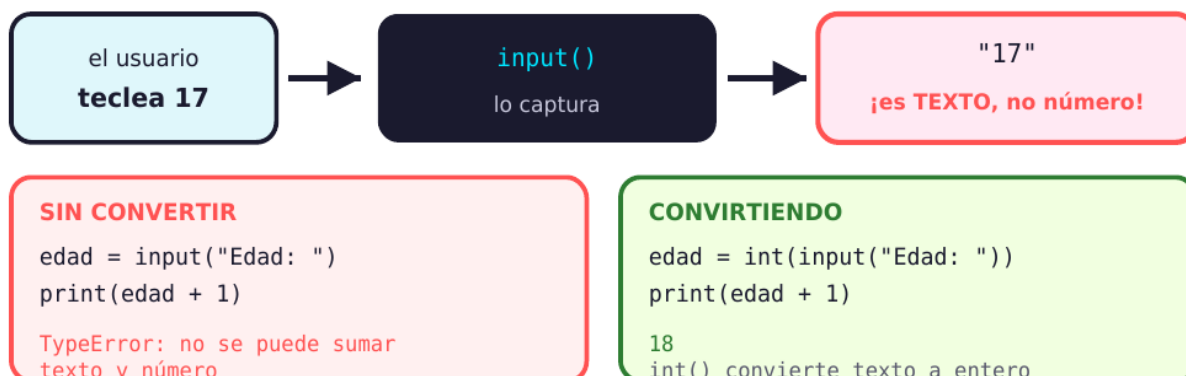


FIG. 4.1 — El origen del error más común. Con textos, el operador + no suma: concatena (pega). Por eso "5" + "3" da "53" y no 8. La solución es convertir el texto a número antes de operar.

4.2 Conversión de tipos (casting)

Convertir un dato de un tipo a otro se llama **casting** y se hace con funciones que llevan el nombre del tipo destino. Míralo funcionando en el caso clásico:

suma_correcta.py

```
# MAL: sin convertir, se pegan como texto
a = input("Primer número: ")
b = input("Segundo número: ")
print(a + b)

# BIEN: convertimos a número entero antes de sumar
a = int(input("Primer número: "))
b = int(input("Segundo número: "))
print(a + b)
```

SALIDA EN PANTALLA

```
Primer número: 5
Segundo número: 3
53    <-- iconcatenó!
Primer número: 5
Segundo número: 3
8      <-- ¡ahora sí sumó!
```

Presta atención a la técnica de la segunda versión: `int(input("..."))`. Se lee de adentro hacia afuera: primero se ejecuta `input()` (que devuelve texto), y ese resultado entra directo en `int()` (que lo convierte a entero). Es la forma estándar de pedir números y la vas a escribir cientos de veces.

Función	Convierte a	Ejemplos y advertencias
<code>int()</code>	Número entero	<code>int("17")</code> → 17. Ojo: <code>int("4.5")</code> da ERROR (tiene punto), e <code>int(4.9)</code> → 4 (trunca, no redondea).
<code>float()</code>	Número con decimales	<code>float("4.5")</code> → 4.5. Acepta también enteros: <code>float("7")</code> → 7.0
<code>str()</code>	Texto	<code>str(17)</code> → "17". Útil para concatenar números dentro de un texto.
<code>bool()</code>	Verdadero/falso	<code>bool(0)</code> → False; <code>bool(cualquier otro número)</code> → True; <code>bool("")</code> → False.

ERROR CLÁSICO • VALUEERROR: EL USUARIO ESCRIBIÓ CUALQUIER COSA

Si haces `int(input())` y la persona escribe "hola" (o deja vacío, o escribe 4.5), Python lanza un **ValueError** y el programa se detiene de golpe. Es el caso borde clásico del que hablabas en lógica: los datos inválidos existen y hay que preverlos. Por ahora, la solución de principiante es advertir claramente en el mensaje qué se espera ("Escribe tu edad en números enteros: "). La solución profesional (`try / except`) llega cuando domines lo básico.

4.3 f-strings: la forma moderna de armar textos

Ya viste dos formas de mezclar variables con texto: con comas en `print()` y concatenando con `+`. Ambas funcionan, pero desde Python 3.6 existe algo mucho mejor: las **f-strings** (cadenas formateadas). Se escribe una `f` justo antes de la comilla, y dentro del texto se ponen las variables entre llaves. Compara las tres versiones:

tres_formas.py

```
nombre = "Ana"
edad = 17

# 1. Con comas: funciona, pero no controlas los espacios
print("Hola", nombre, "tienes", edad, "años")

# 2. Concatenando: obliga a convertir con str() y es incómodo
print("Hola " + nombre + ", tienes " + str(edad) + " años")

# 3. Con f-string: se lee igual que la frase final
print(f"Hola {nombre}, tienes {edad} años")
```

SALIDA EN PANTALLA

```
Hola Ana tienes 17 años
Hola Ana, tienes 17 años
Hola Ana, tienes 17 años
```

La tercera versión gana por goleada: **se lee exactamente como el resultado**, no obliga a convertir tipos y admite operaciones dentro de las llaves. Es lo que usan hoy los profesionales, así que apréndela bien desde el principio:

fstrings_poder.py

```
nota1 = 4.0
nota2 = 3.5

# Se pueden hacer cuentas dentro de las llaves
print(f"El promedio es {(nota1 + nota2) / 2}")

# Y también llamar funciones
nombre = "sofía"
print(f"Bienvenida, {nombre.upper()}")
```

SALIDA EN PANTALLA

```
El promedio es 3.75
Bienvenida, SOFÍA
```

4.4 Formatos: que los números se vean decentes

Un promedio que sale como 3.7333333333333334 se ve horrible. Dentro de una f-string puedes agregar **dos puntos y un formato** para controlar cómo se muestra el número. Los tres que más usarás:

formatos.py

```
promedio = 3.7333333333333334
precio = 1250000

print(f"Promedio: {promedio:.2f}") # 2 decimales
print(f"Promedio: {promedio:.1f}") # 1 decimal
print(f"Precio: ${precio:,}") # separador de miles
print(f"Porcentaje: {0.87:.0%}") # como porcentaje

# Alineación: útil para hacer tablas en consola
print(f"|{'Ana':<10}|{4.5:>6}|")
print(f"|{'Sebastián':<10}|{3.2:>6}|")
```


SALIDA EN PANTALLA

```
Promedio: 3.73
Promedio: 3.7
Precio: $1,250,000
Porcentaje: 87%
|Ana      | 4.5|
|Sebastián| 3.2|
```

El más importante con diferencia es `:.2f` (dos decimales): lo usarás para notas, dinero, promedios y prácticamente cualquier número que un humano vaya a leer. Los símbolos `<` y `>` alinean a la izquierda y a la derecha dentro de un ancho fijo, que es como se arman tablas legibles en la consola.

EN LA VIDA REAL · EL FORMATO ES PARTE DEL MENSAJE

Cualquier app de dinero muestra \$1.250.000 y no 1250000.0: alguien programó ese formato. Un precio mal formateado se percibe como error o como estafa, así que esto no es cosmética: es confianza.

Los formularios que odias —los que se borran si te equivocas o no dicen qué formato esperan— son exactamente el problema del `ValueError` sin resolver. Cuando programes, el mensaje de `input()` es tu primera oportunidad de evitarle sufrimiento a alguien.

? ¿SABÍAS QUÉ?

Antes de 2016, dar formato a un texto en Python era un dolor de cabeza con tres sistemas distintos compitiendo (el operador `%`, el método `.format()` y otros). Un desarrollador llamado **Eric V. Smith** propuso las f-strings en la PEP 498, y su argumento central fue casi psicológico: los otros métodos **separan la variable del lugar donde aparece**, obligando al lector a saltar con los ojos de un lado a otro para entender el resultado. Las f-strings ponen la variable exactamente donde va a aparecer. La comunidad las adoptó tan rápido que hoy son, por lejos, la forma recomendada, y las mediciones muestran además que son más rápidas que las alternativas. **Detalle bonito: ganaron por ser más FÁCILES DE LEER, exactamente el valor número uno del Zen de Python.**

RETO DE OBSERVADOR

MISIÓN: "PROGRAMA CONVERSADOR" · 30 MIN

1. Escribe una calculadora de promedio: pide tres notas con `input()`, conviértelas con `float()`, calcula el promedio y muéstralo con f-string y dos decimales.
2. Haz una "ficha de perfil": pide nombre, edad y ciudad, y devuelve un párrafo bonito con f-strings. Incluye una cuenta dentro de las llaves (por ejemplo, en qué año cumplirá 30).
3. Rómpelo: en tu calculadora, escribe "hola" cuando pida una nota. Copia el error completo. Luego escribe 4,5 con coma en vez de punto y observa qué pasa. Explica ambos casos.
4. **Nivel jefe:** arma una tabla de 3 estudiantes en consola usando alineación (`<` y `>`) para que las columnas queden perfectamente cuadradas. Estás usando tipografía monoespaciada a propósito: bienvenido al diseño de interfaces de texto.

Recompensa desbloqueada: conversación en dos vías. Tu programa ya pregunta, entiende y responde con estilo.

CHECKPOINT • NIVEL 4

1. ¿Por qué `int(input())` es tan común? ¿Qué hace cada función y en qué orden se ejecutan?
2. Explica por qué `"5" + "3"` da `"53"` y cómo se arregla.
3. Escribe de memoria una f-string que muestre un promedio con dos decimales.

ESTE NIVEL EN 3 IDEAS

1. `input()` siempre devuelve texto: convertir con `int()` o `float()` es obligatorio para calcular.
2. Las f-strings son la forma moderna: `f"texto {variable}"`, legible y sin conversiones manuales.
3. Los formatos como `:.2f` y `;`, hacen que tu salida se vea profesional y confiable.

OPERADORES

Calcular, comparar y combinar condiciones

LO QUE DESBLOQUEARÁS

- ▶ Clasificar los operadores en sus 5 familias y saber para qué sirve cada una.
- ▶ Dominar `//` y `%` (división entera y residuo): más útiles de lo que parecen.
- ▶ Combinar condiciones con `and`, `or` y `not` sin equivocarte.
- ▶ Entender la precedencia: quién se ejecuta primero.

PALABRAS CLAVE

operador	operando	aritméticos
comparación	lógicos	asignación
pertenencia	módulo %	precedencia

5.1 Las 5 familias de operadores

Un **operador** es un símbolo que realiza una operación sobre uno o más valores (llamados operandos). Ya usaste varios sin pensarlo: el `=` para asignar y el `+` para sumar. Aquí está el mapa completo de las familias que necesitas:

ARITMÉTICOS · hacen cuentas <code>+</code> <code>-</code> <code>*</code> <code>/</code> suma, resta, multiplica, divide <code>//</code> <code>%</code> división entera y residuo <code>**</code> potencia: <code>2**3</code> es 8	COMPARACIÓN · dan True o False <code>==</code> <code>!=</code> ¿igual? ¿distinto? <code>></code> <code><</code> mayor que, menor que <code>>=</code> <code><=</code> mayor/menor o igual
LÓGICOS · combinan condiciones <code>and</code> las DOS deben cumplirse <code>or</code> basta con UNA <code>not</code> invierte el resultado	ASIGNACIÓN Y PERTENENCIA <code>=</code> <code>+=</code> <code>-=</code> guardar y actualizar <code>in</code> ¿está dentro de? <code>is</code> ¿es exactamente el mismo?

Ojo: `=` guarda un valor · `==` pregunta si dos valores son iguales. El error nº1 de todo principiante.

FIG. 5.1 — El mapa de operadores. Los aritméticos producen números; los de comparación y los lógicos producen True o False (y por eso son la materia prima de las decisiones del Nivel 6).

5.2 Aritméticos: los que hacen cuentas

aritmeticos.py

```
a = 17
b = 5

print(f'Suma:      {a + b}')
print(f'Resta:     {a - b}')
print(f'Multiplicación: {a * b}')
print(f'División:    {a / b}')    # SIEMPRE da float
print(f'División entera: {a // b}') # se queda con la parte entera
print(f'Residuo:     {a % b}')    # lo que sobra de la división
print(f'Potencia:    {a ** 2}')    # 17 al cuadrado
```

SALIDA EN PANTALLA

```
Suma:      22
Resta:     12
Multiplicación: 85
División:  3.4
División entera: 3
Residuo:   2
Potencia:  289
```

Dos advertencias que ahorran horas de confusión. Primera: **la división normal (/) SIEMPRE devuelve un float**, aunque el resultado sea exacto: $10 / 2$ da 5.0, no 5. Segunda: la potencia se escribe con dos asteriscos (**), no con el símbolo ^ que quizás esperabas (ese hace otra cosa completamente distinta en Python).

EL OPERADOR MÁS SUBESTIMADO: EL MÓDULO (%)

El % devuelve el RESIDUO de una división: lo que sobra. $17 \% 5$ es 2 porque 5 cabe tres veces en 17 y sobran 2. Parece inútil hasta que descubres para qué sirve de verdad:

- **Saber si un número es par:** si $n \% 2 == 0$, es par. Si da 1, es impar. Este truco lo usarás toda tu vida.
- **Saber si algo es múltiplo de otra cosa:** $n \% 5 == 0$ significa "n es múltiplo de 5".
- **Trabajar con ciclos que dan la vuelta:** horas de un reloj, días de la semana, turnos que se repiten. Es la matemática de todo lo circular.

modulo_util.py

```
numero = int(input("Escribe un número: "))
print(f"¿Es par? {numero % 2 == 0}")
print(f"¿Es múltiplo de 5? {numero % 5 == 0}")

# Un reloj de 24h: la hora 27 en realidad es la hora 3
print(f"27 horas después de medianoche son las {27 % 24}")
```

SALIDA EN PANTALLA

```
Escribe un número: 12
¿Es par? True
¿Es múltiplo de 5? False
27 horas después de medianoche son las 3
```

5.3 Comparación: los que producen True o False

Estos operadores no calculan: **preguntan**. Y su respuesta siempre es un booleano (True o False), que es justo lo que necesita un if para decidir. Aquí está el error número uno de todo principiante del planeta:

ERROR CLÁSICO · = NO ES LO MISMO QUE ==

Un solo signo igual (=) ASIGNA: guarda un valor en una variable. $edad = 18$ significa "guarda 18 en edad".

Dos signos iguales (==) COMPARAN: preguntan si dos valores son iguales y devuelven True o False. $edad == 18$ significa "¿la edad es 18?".

Truco para no olvidarlo: cuando quieras PREGUNTAR, escribe el doble. Si escribes uno solo dentro de un if, Python te lanzará un SyntaxError, y ahora ya sabrás exactamente qué mirar.

comparacion.py

```
nota = 4.2

print(nota == 4.2) # ¿es igual a 4.2?
print(nota != 3.0) # ¿es distinta de 3.0?
print(nota > 4.5)  # ¿es mayor que 4.5?
print(nota >= 3.0) # ¿es mayor o igual a 3.0?

# Con textos también funciona (compara alfabéticamente)
print("ana" == "Ana") # ojo: las mayúsculas importan
```

SALIDA EN PANTALLA

```
True
True
False
True
False
```

5.4 Lógicos: combinar varias preguntas

Cuando una condición no basta, los operadores lógicos unen varias. La forma más clara de entenderlos es una tabla de verdad, pero en español sencillo: **and exige que TODO se cumpla; or se conforma con que UNO se cumpla; not le da la vuelta a la respuesta.**

logicos.py

```
edad = 17
tiene_permiso = True

# and: las dos condiciones deben ser verdaderas
print(edad >= 18 and tiene_permiso)

# or: basta con que una sea verdadera
print(edad >= 18 or tiene_permiso)

# not: invierte el resultado
print(not tiene_permiso)

# Python permite comparaciones encadenadas (¡y se leen preciosos!)
nota = 4.2
print(3.0 <= nota <= 5.0) # ¿está entre 3.0 y 5.0?
```

SALIDA EN PANTALLA

```
False
True
False
True
```

Esa última línea es una joya de Python que casi ningún otro lenguaje tiene: **3.0 <= nota <= 5.0** se escribe igual que en matemáticas. En la mayoría de los lenguajes tendrías que escribir "nota >= 3.0 and nota <= 5.0". Otro punto para la legibilidad.

5.5 Asignación compuesta y precedencia

asignacion.py

```
puntos = 100

puntos = puntos + 50  # forma larga
puntos += 50          # forma corta: exactamente lo mismo
puntos -= 30          # restar y guardar
puntos *= 2           # multiplicar y guardar

print(puntos)

# Pertenencia: ¿está dentro?
frase = "me encanta programar"
print("gram" in frase)
```

SALIDA EN PANTALLA

```
340
True
```

Por último, la **precedencia**: cuando hay varios operadores en la misma línea, Python respeta el mismo orden que aprendiste en matemáticas. Primero los paréntesis, luego las potencias, después multiplicación y división, luego suma y resta, después las comparaciones y de últimos los lógicos (not, después and, después or). Consejo profesional que vale más que memorizar la tabla: **si dudas, usa paréntesis**. No cuestan nada y hacen tu código legible para todos, incluido tu yo del futuro.

precedencia.py

```
print(2 + 3 * 4)      # multiplica primero: 14
print((2 + 3) * 4)    # los paréntesis mandan: 20

edad = 20
tiene_entrada = False
es_socio = True

# ¿and o or primero? Mejor no adivinar: usa paréntesis
print(edad >= 18 and (tiene_entrada or es_socio))
```

SALIDA EN PANTALLA

```
14
20
True
```

EN LA VIDA REAL · OPERADORES EN LAS APPS QUE USAS

El filtro de una tienda es un montón de operadores lógicos: `precio >= 50000 and precio <= 200000 and talla == "M" and (color == "negro" or color == "blanco")`. Cada casilla que marcas agrega una condición a esa gran expresión.

El módulo % en tu vida diaria: las filas alternadas de colores en una tabla, los turnos rotativos, los repartos de cartas en un juego, "cada 3 puntos ganas una vida". Todo lo que se repite en ciclos usa el residuo.

? ¿SABÍAS QUÉ?

El símbolo % se llama técnicamente **operador módulo** y tiene una aplicación que sostiene la seguridad de internet: la **aritmética modular** es la base matemática de la criptografía moderna. Cuando tu celular se conecta a una página segura, se ejecutan operaciones con residuos sobre números gigantescos, porque tienen una propiedad mágica: son fáciles de calcular en un sentido y prácticamente imposibles de revertir. También lo usan los dígitos de verificación de las tarjetas de crédito y de los documentos de identidad: un algoritmo calcula un residuo y lo compara con el último dígito para detectar errores de tipeo al instante. **El operador que te enseñan para saber si un número es par es, literalmente, uno de los pilares que protege tu dinero.**

RETO DE OBSERVADOR

MISIÓN: "LA CALCULADORA TOTAL" · 30 MIN

1. Escribe un programa que pida dos números y muestre las 7 operaciones aritméticas de 5.2 con f-strings alineadas. Prueba con 17 y 5 y verifica cada resultado a mano.
2. Programa un "detector de números": pide un número y muestra si es par, si es múltiplo de 3 y si está entre 1 y 100 (usa la comparación encadenada).
3. Simula el filtro de una tienda: crea variables (precio, talla, color, hay_stock) y escribe UNA sola expresión lógica que diga si el producto cumple lo que buscas. Prueba cambiando valores.
4. **Nivel jefe:** predice a mano el resultado de estas tres líneas ANTES de ejecutarlas: $10 / 2$, $10 // 3$, $2 ** 3 ** 2$. Luego ejecútalas. Si la tercera te sorprendió, investiga por qué (pista: no todos los operadores se leen de izquierda a derecha).

Recompensa desbloqueada: caja de herramientas completa. Ya puedes calcular, comparar y combinar: justo lo que necesita el nivel siguiente.

CHECKPOINT · NIVEL 5

1. Explica la diferencia entre $/$, $//$ y $\%$, con un ejemplo de cada uno.
2. ¿Cuál es la diferencia entre $=$ y $==$? ¿Por qué es el error más común?
3. Traduce a español: `edad >= 18 and (tiene_entrada or es_socio)`

ESTE NIVEL EN 3 IDEAS

1. **Cinco familias:** aritméticos, comparación, lógicos, asignación y pertenencia.
2. **Los de comparación y los lógicos producen True/False:** son el combustible de las decisiones.
3. **Ante la duda de precedencia, paréntesis:** cuestan cero y multiplican la claridad.

DECISIONES: IF, ELIF, TERNARIO Y MATCH

Que tu programa elija caminos según la situación

LO QUE DESBLOQUEARÁS

- ▶ Escribir decisiones con `if` / `elif` / `else` y entender la indentación.
- ▶ Encadenar y anidar condiciones sin perderte.
- ▶ Usar el operador ternario para decidir en una sola línea.
- ▶ Dominar `match-case`, la estructura moderna para muchos casos.

PALABRAS CLAVE

<code>if</code>	<code>elif</code>	<code>else</code>	indentación	bloque
condición	ternario	<code>match</code>	<code>case</code>	comodín

6.1 if: el primer cruce de caminos

Hasta ahora tus programas iban en línea recta. Con **if** aparecen las bifurcaciones: un bloque de código que solo se ejecuta SI se cumple una condición. La estructura tiene tres partes obligatorias y una regla de oro:

primer_if.py

```
edad = int(input("¿Cuántos años tienes? "))

if edad >= 18:
    print("Eres mayor de edad")
    print("Puedes votar")

print("Fin del programa")
```

SALIDA EN PANTALLA

```
¿Cuántos años tienes? 20
Eres mayor de edad
Puedes votar
Fin del programa
```

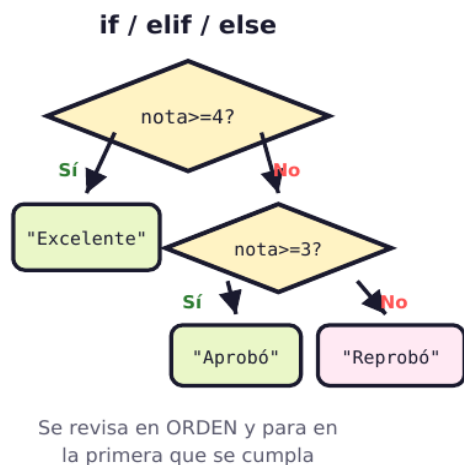
LAS 3 PARTES DE UN IF (Y EL ERROR QUE ARRUINA EL PRIMER DÍA)

- 1. La palabra `if` y la condición:** una expresión que da `True` o `False` (aquí entran todos los operadores del Nivel 5).
- 2. Los dos puntos (`:`) al final:** son obligatorios y significan "aquí empieza el bloque". Olvidarlos es el error más común: `SyntaxError`.
- 3. La indentación del bloque:** las líneas de adentro van corridas 4 espacios. Esos espacios NO son estilo: son lo que le dice a Python qué está dentro del `if`. Es la regla de oro del lenguaje.

Comprueba tú mismo el poder de la indentación mirando el ejemplo anterior: las dos primeras líneas están corridas, así que solo se ejecutan si eres mayor de edad; **"Fin del programa" no está corrida, así que se ejecuta SIEMPRE**. Cambia la sangría y cambias el significado del programa. En otros lenguajes, mover un espacio no altera nada; en Python, lo altera todo.

6.2 else y elif: la cadena de decisiones

Con **else** defines qué pasa cuando la condición NO se cumple, y con **elif** (contracción de "else if") agregas condiciones intermedias. Puedes poner tantos elif como quieras, pero solo un if al principio y como mucho un else al final:



EL TERNARIO (una línea)

```
estado = "Aprobó" if n >= 3 else ...
```

Solo para decisiones MUY simples de dos caminos.

match / case (Python 3.10+)

```
match dia:
case "lunes" : ...
case "martes" : ...
case _ : ... (_ = cualquier otro)
```

Ideal cuando comparas UN valor contra muchas opciones fijas: menús, comandos, días.

FIG. 6.1 — Las tres formas de decidir. La cadena if/elif/else se evalúa de arriba abajo y se detiene en la primera condición verdadera; el ternario elige entre dos valores en una línea; match compara una variable contra muchos casos.

calificador.py

```
nota = float(input("Escribe tu nota (0.0 a 5.0): "))
```

```
if nota >= 4.5:
    print("¡Excelente! 🏆")
elif nota >= 4.0:
    print("Muy bien")
elif nota >= 3.0:
    print("Aprobaste")
else:
    print("A recuperar")

print(f"Tu nota fue {nota:.1f}")
```

SALIDA EN PANTALLA

```
Escribe tu nota (0.0 a 5.0): 4.2
Muy bien
Tu nota fue 4.2
```

Hay un detalle enorme escondido aquí: **la cadena se detiene en el PRIMER caso verdadero**. Con nota 4.2, Python revisa " $4.2 \geq 4.5$ " (falso), luego " $4.2 \geq 4.0$ " (verdadero), ejecuta ese bloque y **se salta todo lo demás**, aunque 4.2 también sea mayor que 3.0. Por eso el orden importa muchísimo: si pusieras el elif de 3.0 primero, todas las notas buenas caerían ahí y jamás verías "Excelente".

ERROR CLÁSICO · EL ORDEN MAL PUESTO (BUG SILENCIOSO)

Este código no da error... y está mal: si escribes `if nota >= 3.0` primero y luego `elif nota >= 4.5`, la segunda condición **nunca se ejecutará**, porque toda nota mayor a 4.5 ya cumplió la primera. El programa funciona, no se queja y hace algo incorrecto. Es el tipo de bug que solo caza una prueba de escritorio: ejecuta a mano con nota 5.0 y observa por dónde pasa.

6.3 Condiciones anidadas y combinadas

Puedes poner un if dentro de otro (anidar) o combinar condiciones con and/or. Casi siempre lo segundo es más legible, pero anidar sirve cuando hay una condición "principal" y otras que solo tienen sentido dentro de ella:

anidado_vs_combinado.py

```
edad = 16
tiene_permiso = True

# Versión anidada: un if dentro de otro
if edad >= 15:
    if tiene_permiso:
        print("Puede entrar")
    else:
        print("Necesita permiso firmado")
else:
    print("Edad insuficiente")

# Versión combinada: más corta y más clara
if edad >= 15 and tiene_permiso:
    print("Puede entrar")
```

SALIDA EN PANTALLA

```
Puede entrar
Puede entrar
```

6.4 El operador ternario: decidir en una línea

Cuando la decisión es simple —elegir entre DOS valores— existe una forma compacta llamada **operador ternario**. Se escribe al revés de como suena: primero el resultado, luego la condición, luego la alternativa. Se lee sorprendentemente bien en voz alta:

ternario.py

```
nota = 4.2

# Forma larga: 4 líneas
if nota >= 3.0:
    estado = "APROBADO"
else:
    estado = "REPROBADO"

# Forma ternaria: 1 línea, exactamente lo mismo
estado = "APROBADO" if nota >= 3.0 else "REPROBADO"

print(estado)

# También sirve directamente dentro de una f-string
edad = 17
print(f'Eres {'mayor' if edad >= 18 else 'menor'} de edad')
```

SALIDA EN PANTALLA

```
APROBADO
Eres menor de edad
```

Advertencia de estilo, importante: el ternario es elegante para casos simples, pero **no lo uses para decisiones complicadas**. Un ternario dentro de otro ternario es ilegible, y ya sabes cuál es el valor número uno de este lenguaje. Si no cabe cómodo en una línea, usa el if normal.

6.5 match-case: cuando hay muchos casos exactos

Cuando necesitas comparar UNA variable contra muchos valores posibles, una cadena de elif se vuelve repetitiva. Desde **Python 3.10** existe la estructura **match**, que hace lo mismo de forma mucho más limpia:

match_basico.py

```
dia = input("Escribe un día (lunes a domingo): ").lower()

match dia:
    case "lunes":
        print("Empieza la semana 🤖")
    case "viernes":
        print("¡Casi libre!")
    case "sábado" | "domingo":
        print("Fin de semana 🍷")
    case _:
        print("Día normal de semana")
```

SALIDA EN PANTALLA

```
Escribe un día (lunes a domingo): sábado
Fin de semana 🍷
```

LAS CLAVES DE MATCH

- **match variable:** indica qué valor vas a comparar. Termina en dos puntos, igual que el if.
- **case valor:** cada camino posible. Solo se ejecuta el primero que coincida, y luego se sale (no hace falta escribir nada para cortar).
- **La barra vertical | significa "o":** case "sábado" | "domingo" atrapa los dos valores con un solo caso.
- **case _ es el comodín:** el guion bajo significa "cualquier otra cosa". Cumple el papel del else y conviene ponerlo siempre al final para que ningún valor quede sin respuesta.

También puedes agregar condiciones extra a un case con la palabra **if** (se llama "guarda"), lo que hace a match sorprendentemente potente:

match_menu.py

```
print("1. Sumar  2. Restar  3. Salir")
opcion = input("Elige una opción: ")

match opcion:
    case "1":
        a = float(input("a: "))
        b = float(input("b: "))
        print(f"Resultado: {a + b}")
    case "2":
        a = float(input("a: "))
        b = float(input("b: "))
        print(f"Resultado: {a - b}")
    case "3":
        print("¡Hasta luego!")
    case _:
        print("Opción no válida")
```

SALIDA EN PANTALLA

1. Sumar 2. Restar 3. Salir

Elige una opción: 1

a: 7

b: 3

Resultado: 10.0

¿Cuándo uso cada uno?

Situación	Usa esto	Por qué
Comparar rangos (mayor que, entre...)	if / elif / else	match compara valores exactos; los rangos son territorio del if.
Elegir entre dos valores simples	Ternario	Una línea limpia en vez de cuatro. Solo si es simple.
Una variable contra muchos valores exactos	match / case	Menús, comandos, días, códigos: más legible que 6 elif seguidos.
Condiciones combinadas complejas	if con and / or	La flexibilidad total sigue siendo del if.

EN LA VIDA REAL · DECISIONES EJECUTÁNDOSE A TU ALREDEDOR

Cada notificación que recibes pasó por un if: si el usuario tiene la app abierta, no mostrar alerta; si es de noche y activó el modo silencio, esperar; si el mensaje es de alguien silenciado, ignorar. Un árbol de decisiones que alguien programó pensando en ti.

Los menús de los videojuegos y los sistemas de "escribe 1 para pedidos, 2 para reclamos" son match-case puro: una entrada, muchos casos exactos, un comodín para lo inesperado.

Y los filtros de contenido: si el video tiene edad recomendada mayor a la del usuario y no hay confirmación del acompañante, entonces bloquear. Decisiones encadenadas exactamente como las que acabas de escribir.

? ¿SABÍAS QUÉ?

Python vivió **casi treinta años sin una estructura tipo switch**, algo que casi todos los demás lenguajes tenían desde el principio. No fue olvido: fue una decisión deliberada. Guido van Rossum llegó a rechazar propuestas argumentando que una cadena de if/elif ya hacía el trabajo y que agregar otra forma de hacer lo mismo iba contra el Zen de Python ("debería haber una manera obvia de hacerlo"). La estructura match llegó por fin en 2021 con Python 3.10... y no es un switch cualquiera: es **coincidencia de patrones** (pattern matching), una idea tomada de los lenguajes funcionales que permite comparar no solo valores, sino la FORMA de los datos: puede desarmar listas y diccionarios dentro del propio case. **Moraleja: esperaron treinta años... y en vez de copiar lo que tenían los demás, trajeron algo más poderoso.**

RETO DE OBSERVADOR

MISIÓN: "EL PROGRAMA QUE ELIGE" · 35 MIN

1. Escribe un clasificador de edades: niño (0-11), adolescente (12-17), adulto (18-64) y adulto mayor (65+). Usa if/elif/else y prueba con al menos un valor de cada rango, incluidos los bordes exactos (11, 12, 17, 18).
2. Convierte esta decisión a ternario: si la temperatura es mayor a 25 el mensaje es "Hace calor", si no "Está fresco". Escríbelo también dentro de una f-string.
3. Haz un menú con match-case de 4 opciones (por ejemplo: consultar nota, ver promedio, ayuda, salir), incluyendo el comodín case _ para opciones no válidas.
4. **Nivel jefe:** toma el código del recuadro rojo (el del orden mal puesto), escríbelo tal cual y hazle una prueba de escritorio con nota 5.0 para demostrar el bug. Luego corrígelo y explica en una frase qué regla del elif habías violado.

Recompensa desbloqueada: libre albedrío. Tus programas dejaron de ser recetas fijas: ahora responden distinto según la situación.

CHECKPOINT · NIVEL 6

1. ¿Por qué la indentación en Python no es estética? Da un ejemplo de cómo cambia el significado.
2. ¿Por qué el orden de los elif es tan importante? Explica con la cadena de notas.
3. ¿Cuándo conviene match en vez de una cadena de elif, y cuándo no?

ESTE NIVEL EN 3 IDEAS

1. **if/elif/else se evalúa de arriba abajo y ejecuta SOLO el primer caso verdadero: el orden es parte de la lógica.**
2. **La indentación define los bloques: mover un espacio cambia qué se ejecuta y qué no.**
3. **El ternario decide entre dos valores en una línea; match compara una variable contra muchos casos exactos.**

ERRORES, DEPURACIÓN Y PROYECTO

Leer errores sin miedo y construir tu primer programa completo

LO QUE DESBLOQUEARÁS

- ▶ Distinguir los 3 tipos de error y saber qué hacer con cada uno.
- ▶ Leer un traceback de Python y encontrar la línea culpable.
- ▶ Depurar con prints y con prueba de escritorio.
- ▶ Construir un proyecto final que use TODO lo aprendido.

PALABRAS CLAVE

SyntaxError IndentationError NameError
 TypeError ValueError traceback error
 lógico depurar PEP 8

7.1 Los 3 tipos de error (y cuál es el peligroso)

Vas a equivocarte muchísimo, y eso está perfecto: **los errores no son fracasos, son información**. Pero no todos son iguales, y saber distinguirlos cambia por completo cómo los enfrentas:

Tipo de error	Cuándo aparece	Cómo se siente
1. De sintaxis	Antes de ejecutar: Python ni siquiera arranca.	Frustrante pero fácil: te dice la línea. Falta un ;, una comilla o un paréntesis.
2. En tiempo de ejecución	El programa arranca y se rompe a mitad de camino.	Te muestra un traceback con el nombre del error. Molesto, pero informativo.
3. Lógico	Nunca: el programa corre feliz... y da un resultado incorrecto.	El peligroso de verdad. Python no puede ayudarte: solo TU razonamiento lo caza.

El tercero es el que separa a un principiante de alguien confiable. Un promedio calculado con la fórmula equivocada no lanza ningún error: simplemente miente. Y la única herramienta contra eso es la que ya conoces del tomo de lógica: **la prueba de escritorio**. Ejecuta a mano, con datos cuyo resultado ya sabes, y compara.

7.2 Leer un traceback sin entrar en pánico

Cuando algo falla, Python imprime un bloque rojo llamado **traceback**. Parece intimidante y en realidad es un mensaje bastante amable, si sabes leerlo **de abajo hacia arriba**:

error_ejemplo.py

```
edad = input("Tu edad: ")
proxima = edad + 1
print(proxima)
```

SALIDA EN PANTALLA

```
Tu edad: 17
Traceback (most recent call last):
  File "error_ejemplo.py", line 2, in <module>
    proxima = edad + 1
TypeError: can only concatenate str (not "int") to str
```

CÓMO SE DESCIFRA ESE MENSAJE

1. La ÚLTIMA línea es la más importante: dice el tipo de error (TypeError) y su explicación. Aquí: "solo se puede concatenar str con str, no int".

2. La línea con "line 2": te dice exactamente dónde ocurrió. Si el archivo es tuyo, ve directo ahí.

3. El diagnóstico: edad es texto (porque viene de input) y le estás sumando un número. La cura la conoces desde el Nivel 4: edad = int(input("Tu edad: ")).

Truco de oro: si un mensaje de error no lo entiendes, cópialo tal cual en un buscador o pídele a una IA que te lo explique. Millones de personas tuvieron ese error antes que tú.

Error frecuente	Qué significa y qué revisar
SyntaxError	Escribiste algo que Python no puede leer. Revisa dos puntos, comillas y paréntesis, en esa línea y en la anterior.
IndentationError	La sangría está mal. Revisa que todo el bloque tenga los mismos espacios (usa siempre 4, nunca mezcles tabs y espacios).
NameError	Usaste una variable que no existe. Casi siempre: la escribiste distinto (nombre vs nombre_) o la usaste antes de crearla.
TypeError	Mezclaste tipos incompatibles. El caso estrella: texto + número.
ValueError	El tipo es correcto pero el valor no sirve: int("hola") o float("4,5") con coma.
ZeroDivisionError	Dividiste entre cero. Protege con un if antes de dividir.

7.3 Depurar: cazar el bug con prints

La técnica más usada del mundo (por principiantes y por veteranos) es sembrar prints para ver qué está pasando por dentro. Mira este bug lógico real y cómo se caza:

```
bug_logico.py

# BUG: siempre dice que reprobó, incluso con notas altas
nota1 = 4.0
nota2 = 5.0
promedio = nota1 + nota2 / 2    # <-- aquí está el error

print(f"DEBUG: promedio vale {promedio}") # print espía

if promedio >= 3.0:
    print("Aprobó")
else:
    print("Reprobó")

SALIDA EN PANTALLA
DEBUG: promedio vale 6.5
Aprobó
```

El print espía delata el problema: el promedio de 4.0 y 5.0 debería ser 4.5, pero dice 6.5. ¿Por qué? **Precedencia** (Nivel 5): Python dividió primero 5.0 entre 2 y después sumó 4.0. La cura son paréntesis: **promedio = (nota1 + nota2) / 2**. Este es el ejemplo perfecto de error lógico: no hubo ningún mensaje rojo, solo un número equivocado esperando arruinarle el semestre a alguien.

PROTOCOLO ANTIPÁNICO · CUANDO ALGO NO FUNCIONA

1. **Lee el error completo, de abajo hacia arriba.** La respuesta suele estar literalmente escrita ahí.
2. **Ve a la línea que indica** (y mira también la anterior).
3. **Imprime las variables sospechosas** con `print(f"DEBUG: x = {x}")` y compara con lo que ESPERABAS.
4. **Si no hay error pero el resultado es raro:** prueba de escritorio con datos cuyo resultado conozcas.
5. **Explícale el problema a alguien (o a un pato de goma).** Verbalizarlo línea por línea hace aparecer el error solo. Es una técnica real y famosa.

7.4 PEP 8: escribir como la comunidad

Python tiene una guía oficial de estilo llamada **PEP 8** que define cómo debe verse el código bien escrito. No es obligatoria para que funcione, pero seguirla te hace entendible para cualquier programador del mundo. Lo esencial: 4 espacios de indentación (nunca tabs), nombres en snake_case, espacios alrededor de los operadores (`x = 5` y no `x=5`), líneas que no se vuelvan kilométricas, y una línea en blanco para separar bloques con sentido propio. **Es la tipografía y el espacio en blanco del código: no cambia lo que dice, cambia si alguien puede leerlo.**

7.5 PROYECTO FINAL: el boletín inteligente

Momento de juntarlo todo. Este programa usa los siete niveles: entrada de datos, conversión, variables, operadores, f-strings con formato, decisiones encadenadas, ternario y match. Escríbelo completo, entiéndelo línea por línea y luego hazlo tuyo:

boletin.py — proyecto final


```

# ===== BOLETÍN INTELIGENTE =====
# Calcula el promedio de 3 notas y entrega un reporte completo

NOTA_MINIMA = 3.0      # constante: no se modifica

print("=" * 34)
print("  BOLETÍN DE CALIFICACIONES")
print("=" * 34)

nombre = input("Nombre del estudiante: ")
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))

promedio = (n1 + n2 + n3) / 3    # ojo con los paréntesis!
mejor = max(n1, n2, n3)
aprobo = promedio >= NOTA_MINIMA  # guarda True o False

# Ternario: elige entre dos textos en una línea
estado = "APROBADO" if aprobo else "REPROBADO"

print(f"\nEstudiante: {nombre.title()}")
print(f"Promedio: {promedio:.2f}")
print(f"Mejor nota: {mejor:.1f}")
print(f"Estado: {estado}")

# Cadena de decisiones: el orden importa
if promedio >= 4.5:
    print("Desempeño: SUPERIOR 🏆")
elif promedio >= 4.0:
    print("Desempeño: ALTO")
elif promedio >= NOTA_MINIMA:
    print("Desempeño: BÁSICO")
else:
    print("Desempeño: BAJO")

# match: recomendación según lo que el estudiante quiera hacer
print("\n¿Qué quieres hacer? (repasar / mejorar / salir)")
accion = input("> ").lower()

match accion:
    case "repasar":
        print("Empieza por la materia de tu nota más baja.")
    case "mejorar":
        falta = round((4.0 * 3) - (n1 + n2 + n3), 2)
        print(f"Para llegar a 4.0 te faltan {falta} puntos en total.")
    case "salir":
        print("¡Hasta la próxima!")
    case _:
        print("No entendí esa opción.")

```

SALIDA EN PANTALLA

```
=====
BOLETÍN DE CALIFICACIONES
=====
Nombre del estudiante: sofía ramírez
Nota 1: 4.5
Nota 2: 3.8
Nota 3: 4.9

Estudiante: Sofía Ramírez
Promedio: 4.40
Mejor nota: 4.9
Estado: APROBADO
Desempeño: ALTO

¿Qué quieres hacer? (repasar / mejorar / salir)
> mejorar
Para llegar a 4.0 te faltan -1.2 puntos en total.
```

Fíjate en la última línea de la salida: el resultado es **-1.2**, un número negativo... porque el estudiante YA superó el 4.0. Técnicamente correcto, pero comunicativamente feo: **acabas de encontrar un caso borde en tu propio proyecto**. Arreglarlo (con un if que muestre un mensaje distinto si ya alcanzó la meta) es tu primer trabajo como programador de verdad, y es exactamente el reto que sigue.

EN LA VIDA REAL · ASÍ TRABAJA ALGUIEN QUE PROGRAMA

Nadie escribe el programa completo de una vez. Se escribe un pedacito, se ejecuta, se verifica, se sigue. Ese ciclo corto —escribir, probar, corregir— es el hábito más valioso que puedes construir, y evita el escenario de terror del principiante: 80 líneas escritas de un tirón y un error que puede estar en cualquier parte.

Los errores son públicos y compartidos: existen comunidades enormes donde cada mensaje de error de Python ya fue preguntado, respondido y explicado mil veces. Buscar tu error no es hacer trampa: es exactamente lo que hacen los profesionales todos los días.

Y sobre la IA: puede escribirte este proyecto en dos segundos. Pero recuerda el semáforo: úsala para que te EXPLIQUE lo que no entiendes, no para saltarte el músculo que estás construyendo. Si aceptas código que no puedes explicar línea por línea, la deuda se cobra sola en el próximo examen o en la primera entrevista.

? ¿SABÍAS QUÉ?

El error más caro de la historia relacionado con la escritura de código fue, probablemente, **un guion**. En 1962, la sonda estadounidense Mariner 1 rumbo a Venus tuvo que ser destruida 293 segundos después del despegue porque se desvió peligrosamente de su trayectoria. La investigación encontró que, al transcribir a mano las fórmulas al código de guiado, se omitió una barra superior (un símbolo que indicaba "usar el valor promedio" de una variable). Sin ese símbolo, el sistema reaccionaba a variaciones normales de velocidad como si fueran fallos graves. El escritor Arthur C. Clarke la llamó "la barra más cara de la historia": el costo rondó los 18 millones de dólares de la época. **Un símbolo. Uno. La próxima vez que Python te reclame por una comilla que falta, recuerda que ese mensaje molesto es en realidad tu red de seguridad.**

RETO DE OBSERVADOR

RETO FINAL: "HAZ TUYO EL PROYECTO" · 45 MIN

1. Escribe el proyecto completo (a mano, sin copiar y pegar) y ejecútalo con tres juegos de datos: notas altas, notas bajas y el caso borde exacto de 3.0.
2. Arregla el bug comunicativo que encontramos: si al estudiante ya le sobran puntos para el 4.0, debe mostrar un mensaje de felicitación en vez de un número negativo. Usa un if o un ternario.
3. Amplíalo con al menos dos mejoras tuyas: número de notas variable, mensaje personalizado por rango, un case nuevo en el match, o protección contra el ValueError. Que sea TU programa.
4. **Jefe final:** introduce a propósito un error de cada uno de los 3 tipos (syntaxis, ejecución y lógico) en tu proyecto, uno por uno, y documenta en tu cuaderno: qué error salió, cómo lo detectaste y cómo lo arreglaste. Ese documento es, literalmente, tu primer manual de depuración.

Recompensa desbloqueada: programador o programadora. Ya no sigues tutoriales: construyes, rompes, diagnosticas y arreglas.

CHECKPOINT · NIVEL FINAL

1. ¿Cuál de los 3 tipos de error es más peligroso y por qué Python no puede ayudarte con él?
2. Explica cómo se lee un traceback y qué línea contiene la información más útil.
3. En el bug del promedio, ¿qué regla del Nivel 5 se había violado y cómo se arregla?

ESTE NIVEL EN 3 IDEAS

1. **Los errores son información: los de syntaxis y ejecución te avisan; los lógicos solo los caza tu razonamiento.**
2. **El traceback se lee de abajo hacia arriba: última línea = tipo de error; línea del medio = dónde ocurrió.**
3. **Escribir un poco, ejecutar, verificar y repetir es el hábito que sostiene toda una carrera.**